ANÁLISIS PREVIO DEL DESARROLLO WEB

Asignatura: Programación

Docente: Prof. Cristian Medin

Proyecto: Sistema de Gestión de Pedidos Online y Panel de Control para Pizzería

Alumno: Kevin Ivan Gamboa

1. Introducción y Objetivos del Proyecto

1.1. Contexto y Problema a Resolver

El presente proyecto consiste en el análisis, diseño e implementación de una aplicación web dinámica enfocada en la gestión integral de un comercio gastronómico real: Tocci's (Pizzeria & Focacceria). Actualmente, la recepción de pedidos por vías informales (llamadas telefónicas o mensajes sueltos de WhatsApp) genera demoras en la atención, errores en la toma de comanda y falta de un registro centralizado. La aplicación web busca digitalizar y estructurar el proceso de venta, permitiendo al cliente armar su pedido de forma interactiva, seleccionar el método de entrega y consultar el estado de preparación de su pedido. Asimismo, provee al establecimiento de una herramienta administrativa privada para gestionar su catálogo de productos, promociones, novedades y estados de comanda.

1.2. Objetivo General

Desarrollar una aplicación web funcional interconectada que implemente el manejo de base de datos relacional (MySQL), procesamiento en servidor (PHP), interactividad en cliente (JavaScript) y una interfaz de usuario adaptativa (HTML5 y CSS3), garantizando validaciones en frontend y backend junto con un sistema de autenticación de usuarios.

1.3. Objetivos Específicos

- **Funcional:** permitir al cliente armar un pedido, elegir modalidad de entrega y medio de pago, y realizar el seguimiento de su estado sin necesidad de registrarse.
- **Administrativo:** ofrecer un panel privado con control de acceso para gestionar pedidos, productos, promociones y novedades.
- **Seguridad:** proteger la aplicación frente a las vulnerabilidades más comunes (inyección SQL, XSS, CSRF, manipulación de precios, carga de archivos maliciosos y fuerza bruta en el login).
- **Usabilidad:** garantizar una interfaz responsive, utilizable desde celulares y computadoras.
- **Mantenibilidad:** separar responsabilidades y organizar el código en módulos reutilizables.

2. Requisitos y Entorno de Desarrollo

2.1. Requisitos Funcionales

- **RF-01:** El sistema debe mostrar un catálogo de productos y promociones categorizados.
- **RF-02:** El cliente debe poder agregar, modificar y quitar productos de un carrito.
- **RF-03:** El cliente debe poder confirmar un pedido como invitado, indicando datos de contacto, modalidad de entrega y medio de pago.
- **RF-04:** El sistema debe generar un número de pedido único y ofrecer el contacto por WhatsApp para coordinar el pago.
- **RF-05:** El cliente debe poder consultar el estado de su pedido en tiempo real.
- **RF-06:** El cliente debe poder dejar y leer reseñas con puntuación.
- **RF-07:** El cliente debe poder enviar consultas mediante un formulario de contacto.
- **RF-08:** El administrador debe poder iniciar sesión y gestionar pedidos, productos, promociones, noticias y reseñas.

2.2. Requisitos No Funcionales

- **RNF-01 (Seguridad):** Consultas preparadas (PDO), sanitización de salidas, contraseñas cifradas con bcrypt, protección CSRF y control estricto de sesiones.
- **RNF-02 (Usabilidad):** Diseños adaptables a distintos tamaños de pantalla (Mobile First).
- **RNF-03 (Rendimiento):** Imágenes optimizadas e índices en las columnas de búsqueda frecuente en la base de datos.
- **RNF-04 (Mantenibilidad):** Código organizado por responsabilidades, con nombres consistentes y archivos modulares.
- **RNF-05 (Compatibilidad):** Funcionamiento correcto en navegadores modernos (Chrome, Firefox, Edge).

2.3. Tecnologías y Entorno

- **Servidor local:** XAMPP (Apache + MySQL/MariaDB + PHP).
- **Lenguaje de servidor:** PHP 8.x.
- **Base de datos:** MySQL 8.x, acceso mediante PDO.
- **Cliente:** HTML5, CSS3 y JavaScript (ES6).
- **Control de versiones:** Git y GitHub.
- **Editor de código:** Visual Studio Code.

3. Arquitectura de Software y Estructura de Directorios

El desarrollo respetará la separación de responsabilidades y la organización en subcarpetas para garantizar la mantenibilidad del código y la correcta gestión de recursos estáticos.

3.1. Estructura de Archivos y Directorios

Tocci's (Pizzeria & Focacceria)

```
├── css/
│   └── estilos.css          # Hoja de estilos global (layout, tipografías, componentes)
├── js/
│   ├── carrito.js           # Lógica del carrito, subtotales y localStorage
│   ├── validaciones.js      # Validaciones de formularios en cliente
│   └── seguimiento.js       # Actualización periódica del estado del pedido (fetch)
├── img/
│   ├── productos/           # Fotografías optimizadas de pizzas, empanadas y bebidas
│   ├── noticias/            # Imágenes para las publicaciones del blog / historia
│   └── banners/             # Imágenes institucionales y promocionales
├── includes/
│   ├── conexion.php         # Conexión a la base de datos MySQL (PDO)
│   ├── header.php           # Encabezado y menú de navegación
│   ├── footer.php           # Pie de página unificado
│   ├── auth.php             # Verificación de sesión y control de acceso
│   └── funciones.php        # Helpers: sanitización, token CSRF y funciones generales
├── db/
│   └── schema.sql           # Script SQL de creación de la base de datos y datos iniciales
├── index.php                # Vista principal / Inicio
├── nosotros.php             # Historia del local y blog de novedades
├── menu.php                 # Catálogo interactivo de productos y promociones
├── carrito.php              # Vista del carrito de compras
├── checkout.php             # Confirmación de pedido y procesamiento en PHP
├── seguimiento.php          # Consulta de estado del pedido
├── resenas.php              # Calificaciones y opiniones de clientes
├── contacto.php             # Formulario institucional de contacto
├── login.php                # Inicio de sesión administrativo
├── logout.php               # Cierre de sesión
└── admin.php                # Panel de control privado (gestión del sistema)
```

3.2. Decisiones de Arquitectura

- **Separación de responsabilidades:** `auth.php` se ocupa exclusivamente de la verificación de sesiones, mientras que `funciones.php` concentra la sanitización de datos y generación/validación de tokens CSRF.
- **Reutilización de componentes:** `header.php` y `footer.php` se incluyen en todas las vistas públicas para mantener consistencia visual y no repetir código.
- **Gestión de recursos estáticos:** Las imágenes se organizan por carpetas según su uso (productos, noticias, banners) para mantener el proyecto limpio y ordenado.

4. Descripción de las Vistas

4.1. index.php (Inicio)

- **Función:** página de bienvenida y presentación comercial del establecimiento.
- **Contenido:** banner principal, promociones destacadas obtenidas dinámicamente desde la base de datos, accesos directos al menú y adelanto de las últimas novedades.

4.2. nosotros.php (Historia y Blog de Novedades)

- **Función:** reseña institucional del origen de la pizzería y canal de comunicación tipo blog.
- **Contenido:** historia y valores del comercio, fotografías del equipo o local, y listado dinámico de publicaciones (noticias, promociones especiales, eventos) cargadas por el administrador desde la base de datos.

4.3. menu.php (Menú Interactivo)

- **Función:** catálogo general de productos y promociones.
- **Contenido:** listado de pizzas, empanadas y bebidas categorizadas. Cada producto incluye imagen, descripción, precio unitario (o precio promocional, si corresponde) y botón interactivo "Agregar al carrito". Filtros por categoría mediante JavaScript/PHP. Al agregar un producto, el cliente puede indicar

una aclaración opcional (ejemplo: *"sin cebolla"*), que se conserva en el carrito y se guarda luego en `detalle_pedidos.aclaraciones`.

4.4. carrito.php (Carrito de Compras)

- **Función:** revisión previa de la orden antes de la compra.
- **Contenido:** tabla dinámica que enumera los productos seleccionados, permite modificar cantidades, editar aclaraciones, eliminar ítems y calcula el total estimado. Botón para proceder a la confirmación (`checkout.php`).

Nota: el total mostrado en esta vista es informativo; el importe definitivo lo calcula de manera estricta el servidor.

4.5. checkout.php (Confirmación del Pedido)

- **Función:** captura de datos del cliente, método de entrega y persistencia del pedido (compra como invitado, sin registro previo).
- **Contenido:** formulario donde el usuario ingresa sus datos de contacto (nombre, teléfono, dirección), modalidad (*Delivery / Retiro en local*) y medio de pago. Al enviar:
    - Se transfieren los productos del carrito desde `localStorage` a PHP mediante un campo oculto en formato JSON, que contiene únicamente `producto_id`, `cantidad` y `aclaraciones`.
    - PHP valida el token CSRF y los campos del formulario en el servidor.
    - Por cada ítem, PHP verifica que el producto exista y esté activo, y obtiene el precio real desde la base de datos. Se descartan precios o totales enviados desde el cliente.
    - PHP calcula el total (incluido el costo de envío si aplica) y guarda la orden en MySQL dentro de una transacción con estado inicial `'Pendiente de pago'`.
    - Muestra el número de pedido único generado (ej: `#1052`) y presenta un botón para redirigir a WhatsApp con un mensaje pre completado para coordinar el pago.
    - Se vacía el carrito almacenado en `localStorage`.

4.6. seguimiento.php (Seguimiento de Estado)

- **Función:** consulta del progreso del pedido en tiempo real.
- **Contenido:** formulario de búsqueda que requiere número de pedido y teléfono de manera coincidente (para no exponer datos privados). Ejecuta una consulta preparada y muestra la etapa actual de la comanda (*Pendiente de pago, En preparación, En camino / Listo para retirar, Entregado, Cancelado*). Mediante JavaScript (`js/seguimiento.js`), la vista actualiza el estado periódicamente vía `fetch` sin recargar la página.

4.7. resenas.php (Reseñas y Calificaciones)

- **Función:** módulo público de opinión y retroalimentación de clientes.
- **Contenido:** histórico de comentarios aprobados recibidos desde MySQL con su puntuación (estrellas). Formulario abierto donde cualquier cliente puede dejar su nombre, calificación (1 a 5) y comentario. Incluye un campo *honeypot* e intervalo mínimo entre envíos por sesión/IP para mitigar el spam. Las reseñas requieren aprobación en el panel administrativo (`visible = 1`).

4.8. contacto.php (Formulario de Contacto / Ubicación)

- **Función:** canal de comunicación institucional con controles variados.
- **Contenido:** mapa de ubicación e información de horarios. Formulario con controles HTML variados: `input[type="text"]` (Nombre y Apellido), `input[type="email"]` (Correo), `input[type="tel"]` (Teléfono), `select` (Motivo de consulta), `input[type="radio"]` (Preferencia de respuesta), `input[type="checkbox"]` (Términos/Ofertas) y `textarea` (Mensaje). Incluye protección *honeypot*.

4.9. login.php (Acceso de Administrador)

- **Función:** autenticación de personal autorizado.
- **Contenido:** formulario con usuario y contraseña. Procesa las credenciales en PHP mediante `password_verify()`, regenera el identificador de sesión

con `session_regenerate_id(true)` e inicia la sesión (`$_SESSION`). Ante errores muestra mensajes genéricos (*"Usuario o contraseña incorrectos"*) y limita intentos fallidos consecutivos.

4.10. admin.php (Panel de Control y Gestión)

- **Función:** gestión privada del sistema accesible solo con sesión activa.
- **Contenido:**
  - **Gestión de Pedidos:** listado de comandas con opción de actualizar el estado (*En preparación, En camino / Listo para retirar, Entregado, Cancelado*).
  - **Gestión de Productos y Promociones:** ABM (Alta, Baja y Modificación) de productos del menú. La baja es lógica (`activo = 0`) para preservar la integridad de pedidos históricos.
  - **Gestión del Blog:** publicación de noticias e historias en la sección `nosotros.php`.
  - **Moderación de Reseñas:** aprobación o desactivación de comentarios de clientes.

    Nota de arquitectura: El acceso al panel se valida mediante `includes/auth.php`. La estructura de base de datos mantiene el campo `rol` para permitir la asignación de permisos diferenciados a futuro.

5. Esquema y Modelo de Datos (Base de Datos MySQL)

La base de datos se denomina `pizzeria_db` y utiliza el motor InnoDB con codificación `utf8mb4` para garantizar la integridad referencial mediante claves foráneas.

5.1. Estructura de Tablas

- **`categorias`**: `id` (INT, PK, AUTO_INCREMENT), `nombre` (VARCHAR(50)).
- **`productos`**: `id` (INT, PK, AUTO_INCREMENT), `categoria_id` (INT, FK → `categorias.id`), `nombre` (VARCHAR(100)), `descripcion` (TEXT),

`precio` (DECIMAL(10,2)), `precio_promo` (DECIMAL(10,2), NULL), `es_promocion` (TINYINT(1), DEFAULT 0), `imagen` (VARCHAR(255)), `activo` (TINYINT(1), DEFAULT 1).

- **clientes**: `id` (INT, PK, AUTO_INCREMENT), `nombre` (VARCHAR(100)), `telefono` (VARCHAR(20)), `direccion` (VARCHAR(200)).
- **pedidos**: `id` (PK, AUTO_INCREMENT), `cliente_id` (FK → `clientes.id`), `fecha` (DATETIME), `costo_envio` (DECIMAL(10,2)), `total` (DECIMAL(10,2)), `estado` (ENUM: `'Pendiente de pago'`, `'En preparación'`, `'En camino / Listo para retirar'`, `'Entregado'`, `'Cancelado'`), `metodo_pago` (ENUM), `tipo_entrega` (ENUM).
- **detalle_pedidos**: `id` (PK, AUTO_INCREMENT), `pedido_id` (FK → `pedidos.id`), `producto_id` (FK → `productos.id`), `cantidad` (INT), `precio_unitario` (DECIMAL(10,2)), `aclaraciones` (VARCHAR(255)).
- **resenas**: `id` (PK, AUTO_INCREMENT), `nombre` (VARCHAR(100)), `puntuacion` (TINYINT), `comentario` (TEXT), `fecha` (DATETIME), `visible` (TINYINT(1), DEFAULT 0).
- **contactos**: `id` (PK, AUTO_INCREMENT), `nombre` (VARCHAR(100)), `email` (VARCHAR(100)), `telefono` (VARCHAR(20)), `motivo` (VARCHAR(50)), `medio_respuesta` (VARCHAR(20)), `mensaje` (TEXT), `fecha` (DATETIME).
- **usuarios**: `id` (PK, AUTO_INCREMENT), `usuario` (VARCHAR(50), UNIQUE), `password` (VARCHAR(255)), `nombre` (VARCHAR(100)), `rol` (ENUM: `'admin'`, `'empleado'`).
- **noticias**: `id` (PK, AUTO_INCREMENT), `titulo` (VARCHAR(150)), `contenido` (TEXT), `imagen` (VARCHAR(255)), `fecha` (DATETIME), `usuario_id` (FK → `usuarios.id`).

5.2. Índices de Optimización

- **`clientes.telefono`:** para acelerar la búsqueda de historial y consultas en `seguimiento.php`.
- **`pedidos.estado`:** para agilizar el filtrado de comandas en el panel administrativo.
- **`productos.categoria_id` y `productos.activo`:** para optimizar la carga del menú público.

6. Estrategia de Validaciones y Tratamiento de Datos

6.1. Validaciones en Cliente (JavaScript)

- **Comprobación de formularios:** verificación de campos obligatorios vacíos antes de permitir el envío en `validaciones.js`.
- **Expresiones regulares (RegEx):** validación de formatos válidos para correo electrónico y número telefónico.
- **Control de datos en carrito:** verificación de que las cantidades sean números enteros positivos mayores a cero y persistencia de aclaraciones por producto en `localStorage`.

6.2. Validaciones y Seguridad en Servidor (PHP)

- **Sanitización y Escape:** uso de `trim()`, `filter_var()` y `htmlspecialchars()` en las salidas para prevenir ataques de inyección de scripts (XSS).
- **Consultas Preparadas:** implementación estricta de la interfaz PDO con marcadores contextuales (`prepare()` y `bindValue()`) en todas las operaciones a la base de datos para prevenir inyección SQL (SQLi).
- **Integridad de Precios y Lógica de Negocio:** en `checkout.php`, el servidor no confía en los montos ni subtotales enviados desde el cliente (`localStorage`). PHP toma únicamente los IDs y cantidades, consulta los precios reales en la tabla `productos` y calcula el total final dentro de una transacción en MySQL.

- **Autenticación y Manejo de Sesiones:** cifrado de contraseñas mediante `password_hash()` con algoritmo `bcrypt`. En `login.php` se verifica la credencial con `password_verify()`, se regenera el ID de sesión mediante `session_regenerate_id(true)` para prevenir fijación de sesión y se limita la cantidad de intentos fallidos.
- **Control de Acceso:** inclusión obligatoria de `includes/auth.php` al inicio de `admin.php` para validar la existencia de `$_SESSION`, restringir el acceso a usuarios no autenticados y redirigir a `login.php`.
- **Protección CSRF y Anti-Spam:** generación y validación de tokens aleatorios por sesión para formularios críticos. Uso de campos ocultos *honeypot* e intervalos mínimos entre envíos en `contacto.php` y `resenas.php` para descartar envíos automatizados por bots.